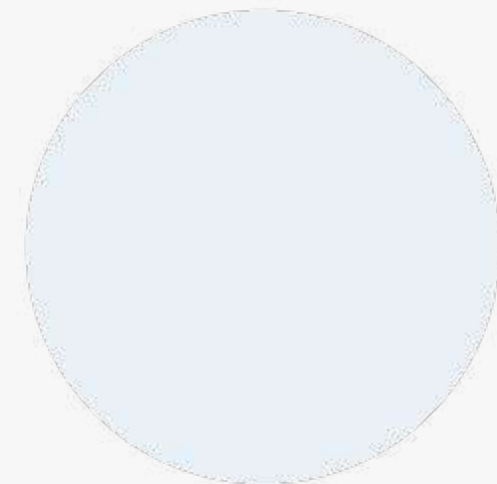


ASTON.



Операторы, циклы, массивы, методы



astondevs.ru

Методы



Общая форма объявления метода выглядит следующим образом:

```
тип_возвращаемого_методом_значения имя_метода  
(список_аргументов) {  
    тело_метода;  
}
```

Тип_возвращаемого_методом_значения обозначает конкретный тип данных (int, float, char, boolean, String и т.д.), возвращаемых методом. Если метод ничего не должен возвращать, указывается ключевое слово void. Для возврата значения из метода используется оператор return.

```
return значение;
```

Для указания имени метода служит идентификатор «имя». Список аргументов обозначает последовательность пар «тип_данных + идентификатор», разделенных запятыми. По существу, аргументы это набор данных, необходимых для работы метода. Если для работы метода не требуются аргументы, то оставляются пустые скобки - ().

Несколько примеров работы с методами:

```
public class FirstApp {  
    public static void main(String[] args) {  
        // для вызова метода необходимо передать ему 2 аргумента типа  
        int,  
        // результатом работы будет целое число, которое напечатается в  
        консоль  
        System.out.println(summ(5, 5));  
        // для вызова метода ему не нужно передавать аргументы,  
        // и он не возвращает данные (метод объявлен как void)  
        printSomeText();  
        // для вызова метода передаем ему в качестве аргумента строку  
        "Java",  
        // которую он выведет в консоль  
        printMyText("Java");  
    }  
    // метод возвращает целое число, принимает на вход два целых числа  
    public static int summ(int a, int b) {  
        // возвращаем сумму чисел return a + b;  
    }  
    // метод ничего не возвращает, не требует входных данных  
    public static void printSomeText() {  
        // печатаем Hello в консоль  
        System.out.println("Hello");  
    }  
    // метод ничего не возвращает, принимает на вход строку  
    public static void printMyText(String txtToPrint) {  
        // выводим строку txtToPrint в консоль  
        System.out.println(txtToPrint);  
    }  
}
```

Методы

При объявлении всех методов внутри основного класса программы после **public** должно идти слово **static**. Если ключевое слово `static` будет отсутствовать, этот метод не получится вызвать из метода `main()`. Смысл этого ключевого слова будет пояснен на следующих занятиях в теме «Объектно-ориентированное программирование».



Несколько примеров работы с методами:

```
public class FirstApp {  
    public static void main(String[] args) {  
        // для вызова метода необходимо передать ему 2 аргумента типа  
        int,  
        // результатом работы будет целое число, которое напечатается в  
        консоль  
        System.out.println(summ(5, 5));  
        // для вызова метода ему не нужно передавать аргументы,  
        // и он не возвращает данные (метод объявлен как void)  
        printSomeText();  
        // для вызова метода передаем ему в качестве аргумента строку  
        "Java",  
        // которую он выведет в консоль  
        printMyText("Java");  
    }  
    // метод возвращает целое число, принимает на вход два целых числа  
    public static int summ(int a, int b) {  
        // возвращаем сумму чисел return a + b;  
    }  
    // метод ничего не возвращает, не требует входных данных  
    public static void printSomeText() {  
        // печатаем Hello в консоль  
        System.out.println("Hello");  
    }  
    // метод ничего не возвращает, принимает на вход строку  
    public static void printMyText(String txtToPrint) {  
        // выводим строку txtToPrint в консоль  
        System.out.println(txtToPrint);  
    }  
}
```

Кодовые блоки



Кодовый блок представляет собой группу операторов. Для оформления в виде блока они помещаются между открывающей и закрывающей фигурными скобками. Созданный кодовый блок становится единым логическим блоком. Такой блок можно использовать при работе с if и for.

```
public static void main(String args[]) { // <-  
    начало кодового блока main  
    int w = 1, h = 2, v = 0;  
    if (w < h) { // <- Начало кодового блока if  
        v = w * h;  
        w = 0;  
    } // <- Конец кодового блока if  
} // <- Конец кодового блока main
```

В данном примере оба оператора в блоке выполняются в том случае, если значение переменной w меньше значения переменной h. Эти операторы составляют единый логический блок, и ни один из них не может быть выполнен без другого. Кодовые блоки позволяют оформлять многие алгоритмы в удобном для восприятия виде.

Области видимости переменных в кодовых блоках:

```
public static void main(String args[]) { //  
    Кодовый блок метода main()  
    int x = 10; // эта переменная доступна для  
    всего кода в методе main  
    if (x == 10) { // Кодовый блок тела if  
        int y = 20; // Эта переменная доступна  
        только в данном кодовом блоке  
        // Обе переменные x и y доступны в данном  
        кодовом блоке  
        System.out.println("x & y: " + x + " " + y);  
        x = y * 2;  
    }  
    // y = 100; // Ошибка! Переменная y недоступна  
    за пределами тела if  
    System.out.println("x = " + x); // Переменная  
    x по-прежнему доступна  
}
```

Операторы



Начнем с базовых понятий. Любая программа представляет собой набор команд, выполняемых компьютером. Чаще всего команды выполняются последовательно, друг за другом. Чуть реже (но все еще довольно часто) возникают ситуации, когда нужно изменить последовательный ход выполнения команд программы. Иногда, в зависимости от некоторых условий, бывает необходимо исполнять один блок команд вместо другого. А когда эти условия меняются, поступать наоборот.

Представим себя на перекрестке семи дорог. Перед нами стоит выбор: свернуть налево или направо, либо же пойти прямо. Наш выбор опирается на некоторые условия. Также у нас нет возможности идти несколькими дорогами одновременно. Т.е. в зависимости от некоторых условий, нам придется выбрать одну дорогу. Такой же принцип и с ветвлением.

Ветвление — это алгоритмическая конструкция, в которой в зависимости от истинности некоторого условия, выполняется одна из нескольких последовательностей действий.

Оператор *if*



Оператор if имеет следующую конструкцию:

```
if (bool_condition) {  
    statement  
}
```

В данной конструкции:

bool_condition — boolean выражение, результатом которого является true или false. Данное выражение называют условием.

statement — команда (может быть не одна), которую необходимо исполнить, в случае, если условие истинно. (bool_statement==true)

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.print("Сколько процентов заряда  
батареи осталось на вашем смартфоне?");  
    int a = scanner.nextInt();  
    if (a < 10) {  
        System.out.println("Осталось менее 10  
процентов, подключите ваш смартфон к зарядному  
устройству");  
    }  
}
```

Это пример простейшей конструкции if. Стоит заметить, что если переменная “a” будет больше либо равна 10, то ничего не произойдет. Программа продолжит выполнять код, который следует за конструкцией if. Также заметим, что в данном случае, у конструкции if есть только одна последовательность действий для исполнения: напечатать текст, либо не делать ничего. Эта вариация ветвления с одной “ветвью”.

Если при обычном if у программы есть выбор: “сделать что-то, либо не делать ничего”, то при if-else выбор программы сводится тому, чтобы “сделать либо одно, либо другое”. Вариант “не делать ничего” отпадает. Вариантов исполнения (или количество ветвей) при таком виде ветвления бывает от двух и более.

```
if (bool_condition) {  
    statement1  
} else {  
    statement2  
}
```

Здесь:

bool_statement — boolean выражение, результатом которого является true или false. Данное выражение называют условием.

statement1 — команда (может быть не одна), которую необходимо выполнить, если условие истинно (bool_statement==true)

statement2 — команда (может быть не одна), которую необходимо выполнить, если условие ложно (bool_statement==false)

Оператор *if*



```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.print("Сколько процентов заряда  
батареи осталось на вашем смартфоне?");  
    int a = scanner.nextInt();  
    if (a < 10) {  
        System.out.println("Осталось менее 10  
процентов, подключите ваш смартфон к зарядному  
устройству");  
    } else {  
        System.out.println("-аряда вашей батареи  
достаточно для того, чтобы прочитать статью на  
Javarush");  
    }  
}
```

Тот же пример об уровне заряда батареи на смартфоне. Только если в прошлый раз программа только лишь предупреждала о необходимости зарядить смартфон, то в этот раз у нас появляется дополнительное уведомление.

Допустимы ситуации, когда после else следуют не команды на исполнение, а еще один if. Тогда конструкция принимает следующий вид:

```
if (bool_condition1) {  
    statement1  
} else if (bool_condition2) {  
    statement2  
} else if (bool_conditionN) {  
    statementN  
} else {  
    statementN+1  
}
```

Оператор *switch*



Оператор switch позволяет делать выбор между несколькими вариантами дальнейшего выполнения программы. Выражение последовательно сравнивается со списком значений оператора switch. При совпадении выполняется набор операторов, связанных с этим условием. Если совпадений не было, выполняется блок default (блок default является необязательной частью оператора switch). Оператор break останавливает выполнение блока case, если break убрать – выполнение кода продолжится дальше.

```
switch (выражение) {  
    case значение1:  
        набор_операторов1;  
        break;  
    case значение 2:  
        набор_операторов2;  
        break;  
    ...  
    default:  
        набор_операторов;  
}
```

Например, последовательность if-else-if-...

```
public static void main(String[] args) {  
    int a = 3;  
    if (a == 1) {  
        System.out.println("a = 1");  
    } else if (a == 3) {  
        System.out.println("a = 3");  
    } else {  
        System.out.println("Ни одно из условий не  
сработало");  
    }  
}
```

Может быть заменена на следующее:

```
public static void main(String[] args) {  
    int a = 3;  
    switch (a) {  
        case 1:  
            System.out.println("a = 1");  
            break;  
        case 3:  
            System.out.println("a = 3");  
            break;  
        default:  
            System.out.println("Ни один из case не  
сработал");  
    }  
}
```


Циклы *for*



Циклы позволяют многократно выполнять последовательность кода.

```
for (инициализация; условие; итерация) {  
    набор_операторов;  
}
```

Инициализация представлена переменной, выполняющей роль счётчика и управляющей циклом (например, `int i = 0;`).
Условие определяет необходимость повторения цикла.
Итерация задаёт шаг изменения переменной, управляющей циклом.

NB: Важно! При написании кода на языке Java обязательно необходимо учитывать регистр символов. (Main не равнозначно main, Void и void не одно и то же, System не равнозначно system и т.д.)

Правильно:

```
for (int i = 0; i < 5; i++) {  
    System.out.println("i = " + i);  
}
```

Неправильно:

```
for (int i = 0, i < 5, i++) {  
    System.out.println("i = " + i);  
}
```

NB: Важно! После закрытой круглой скобки точки с запятой нет. Если вы там ее поставите, цикл будет работать некорректно.

Правильно:

```
for (int i = 0; i < 5; i++) {  
    System.out.println("i = " + i);  
}
```

Неправильно:

```
for (int i = 0; i < 5; i++); {  
    System.out.println("i = " + i);  
}
```

Выполнение цикла for продолжается до тех пор, пока проверка условия дает истинный результат (true).

Циклы *for*



```
public static void main(String args[]) {  
    for (int i = 0; i < 5; i++) {  
        System.out.println("i = " + i);  
    }  
    System.out.println("end");  
}
```

Результат:

```
i=0  
i=1  
i=2  
i=3  
i=4  
end
```

В начале каждого шага цикла проверяется условие $i < 5$. Если это условное выражение верно, вызывается тело цикла (в котором прописан метод `System.out.println(...)`), затем выполняется итерационная часть цикла. Как только условное выражение примет значение `false`, цикл закончит свою работу.

Для управления циклом можно использовать одновременно несколько переменных. В примере ниже за одну итерацию переменная i увеличивается на 1, а j уменьшается на 1.

```
public static void main(String args[]) {  
    for (int i = 0, j = 10; i < j; i++, j--) {  
        System.out.println("i-j: " + i + "-" +  
j);  
    }  
}
```

Результат:

```
i-j: 0-10  
i-j: 1-9  
i-j: 2-8  
i-j: 3-7  
i-j: 4-6
```

При использовании следующей записи цикла `for` можно получить бесконечный цикл. Большинство таких циклов требуют специальное условие для своего завершения.

```
for (;;) {  
    // ...  
}
```

Циклы *for*



Выход из работающего цикла осуществляется оператором `break` без выполнения всего кода из тела цикла, поэтому в результате нет числа 4.

```
public static void main(String[] args) {  
    for(int i = 0; i < 10; i++) {  
        if (i > 3) {  
            break;  
        }  
        System.out.println("i = " + i);  
    }  
}
```

Результат:

i=0

i=1

i=2

i=3

Циклы *foreach*



Еще одной разновидностью цикла for является цикл foreach. Он используется для прохода по всем элементам массива или коллекции, знать индекс проверяемого элемента не нужно. В приведённом ниже примере мы проходим по элементам массива sm типа String и каждому присваиваем временное имя o, то есть «в единицу времени» o указывает на один элемент массива.

```
public static void main(String[] args) {  
    String[] sm = {"A", "B", "C", "D"};  
    for (String o : sm) {  
        System.out.print(o + " ");  
    }  
}
```

Результат:

ABCD

Вложенные циклы



Циклы, работающие внутри других циклов, называют вложенными.

Внимательно разберите последовательность исполнения таких циклов. На всё выполнение внутреннего цикла приходится одна итерация внешнего.

```
public static void main(String args[]) {  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 3; j++) {  
            System.out.print(" " + i + j);  
        }  
    }  
}
```

Результат:

```
00 01 02 10 11 12 20 21 22
```


Циклы *while*



Цикл `while` работает до тех пор, пока указанное условие истинно. Как только условие становится ложным, управление программой передается строке кода, следующей непосредственно после цикла. Если заранее указано условие, которое не выполняется, программа в тело цикла даже не попадет.

```
while (условие) {  
    набор_операторов;  
}
```

Цикл `do-while` очень похож на ранее рассмотренные циклы. В отличие от `for` и `while`, где условие проверялось в самом начале (предусловие), в цикле `do-while` оно проверяется в самом конце (постусловие). Это означает, что цикл `do-while` всегда выполняется хотя бы один раз.

```
do {  
  
    набор_операторов;  
while (условие);
```

Массивы



Для объявления одномерного массива обычно применяется следующая форма.

```
тип_данных[] имя_массива = new тип_данных  
[размер_массива];
```

При создании массива сначала объявляется переменная, ссылающаяся на него. Затем выделяется память для массива, в Java динамически распределяется с помощью оператора `new`; ссылка на неё присваивается переменной. В следующей строке кода создается массив типа `int`, состоящий из 5 элементов, ссылка на него присваивается переменной `arr`.

```
int[] arr = new  
int[5];
```

Массивы



В переменной `arr` сохраняется ссылка на область памяти для массива оператором `new`. Этой памяти должно быть достаточно для размещения в ней 5 элементов типа `int`. Доступ к отдельным элементам массива осуществляется с помощью индексов. Индекс обозначает положение элемента в массиве, индекс первого элемента равен нулю. Если массив `arr` содержит 5 элементов, их индексы находятся в пределах от 0 до 4. Индексирование массива осуществляется по номерам его элементов, заключенным в квадратные скобки. Например, для доступа к первому элементу массива `arr` следует указать `arr[0]`, а для доступа к последнему элементу этого массива — `arr[4]`. В приведенном ниже примере программы в массиве `arr` сохраняются числа от 0 до 4.

Для управления циклом можно использовать одновременно несколько переменных. В примере ниже за одну итерацию переменная `i` увеличивается на 1, а `j` уменьшается на 1.

```
public static void main(String args[]) {  
    int[] arr = new int[5];  
    for(int i = 0; i < 5; i++) {  
        arr[i] = i;  
        System.out.println("arr[" + i + "] = " +  
arr[i]);  
    }  
}
```

Результат:

```
arr[0] = 0  
arr[1] = 1  
arr[2] = 2  
arr[3] = 3  
arr[4] = 4
```

Массивы

Заполнять созданные массивы можно последовательным набором операторов.

```
public static void main(String args[]) {  
    int[] nums = new int[4];  
    nums[0] = 5;  
    nums[1] = 10;  
    nums[2] = 15;  
    nums[3] = 15;  
}
```

В приведённом выше примере массив `nums` заполняется через четыре оператора присваивания. Существует более простой способ решения этой задачи: заполнить массив сразу при его создании.

```
тип_данных[]  
имя_массива = {v1,  
v2, v3, ..., vN} ;
```



Здесь `v1-vN` обозначают первоначальные значения, которые присваиваются элементам массива слева направо по порядку индексирования, при этом Java автоматически выделит достаточный объем памяти.

```
public static void main(String args[]) {  
    int[] nums = { 5, 10, 15, 20 };  
}
```

Границы массива в Java строго соблюдаются. Если обратиться к несуществующему элементу массива, будет получена ошибка.

```
public static void main(String args[]) {  
    int[] arr = new int[10];  
    for(int i = 0; i < 20; i++) {  
        arr[i] = i;  
    }  
}
```

Как только значение переменной `i` достигнет 10, будет сгенерировано исключение `ArrayIndexOutOfBoundsException` и выполнение программы прекратится.

Массивы



Распечатать одномерный массив в консоль можно с помощью конструкции `Arrays.toString()`.

```
import java.util.Arrays;

public class MainClass {
    public static void main(String args[]) {
        String[] arr = {"A", "B", "C", "D"};
        System.out.println(Arrays.toString(arr));
    }
}
```

Результат: [A, B, C, D]

Двумерные массивы



Среди многомерных массивов наиболее простыми являются двумерные. Двумерный массив – это ряд одномерных массивов. При работе с двумерными массивами проще их представлять в виде таблицы, как будет показано ниже. Объявим двумерный целочисленный табличный массив `table` размером 10×20 .

```
int[][] table = new  
int[10][20];
```

Двумерные массивы



В следующем примере создадим двумерный массив размером 3x4, заполним его числами от 1 до 12 и отпечатаем в консоль в виде таблицы.

```
public static void main(String args[]) {  
    int counter = 1;  
    int[][] table = new int[3][4];  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 4; j++) {  
            table[i][j] = counter;  
            System.out.print(table[i][j] + " ");  
            counter++;  
        }  
        System.out.println();  
    }  
}
```

Нерегулярные массивы



Выделяя память под многомерный массив, достаточно указать лишь первый (крайний слева) размер. Память под остальные размеры массива можно выделять по отдельности.

```
int[][] table = new int[3][];  
table[0] = new int[1];  
table[1] = new int[5];  
table[2] = new int[3];
```

Поскольку многомерный массив является массивом массивов, существует возможность установить разную длину массива по каждому индексу. В некоторых случаях такие массивы могут значительно повысить эффективность работы программы и снизить потребление памяти, например, если требуется создать очень большой двумерный массив, в котором используются не все элементы.

Многомерные массивы



В Java допускаются n-мерные массивы, ниже показана форма объявления.

```
тип_данных[][]...[] имя_массива = new тип_данных  
[размер1][размер2]...[размерN];
```

В качестве примера ниже приведено объявление трехмерного целочисленного массива размерами 2x3x4.

```
int[][][] mdarr = new int[2][3][4];
```

Многомерный массив можно инициализировать. Инициализирующую последовательность нужно заключить в отдельные фигурные скобки.

```
тип_данных[][] имя_массива = {  
    { val, val, val, ..., val },  
    { val, val, val, ..., val },  
    { val, val, val, ..., val }  
};
```

Альтернативный синтаксис объявления массивов



Помимо рассмотренной выше общей формы для объявления массива можно также пользоваться следующей формой.

```
тип_данных имя_массива[];
```

Два следующих объявления массивов равнозначны.

```
public static void main(String[] args) {  
    int arr[] = new int[3];  
    int[] arr2 = new int[3];  
}
```


Получение длины массива



При работе с массивами имеется возможность программно узнать его размер. Для этого можно воспользоваться записью `имя_массива.length`. Это удобно использовать, когда нужно пройти циклом `for` по всему массиву.

```
public static void main(String[] args) {  
    int[] arr = {2, 4, 5, 1, 2, 3, 4, 5};  
    System.out.println("arr.length: " + arr.length);  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
}
```

Результат:

arr.length: 8

24512345

Практическое задание



1. Написать метод, принимающий на вход два целых числа и проверяющий, что их сумма лежит в пределах от 10 до 20 (включительно), если да – вернуть true, в противном случае – false.

2. Написать метод, которому в качестве параметра передается целое число, метод должен напечатать в консоль, положительное ли число передали или отрицательное.
Замечание: ноль считаем положительным числом.

3. Написать метод, которому в качестве параметра передается целое число. Метод должен вернуть true, если число отрицательное, и вернуть false если положительное.

4. Написать метод, которому в качестве аргументов передается строка и число, метод должен отпечатать в консоль указанную строку, указанное количество раз;

5. Написать метод, который определяет, является ли год високосным, и возвращает boolean (високосный - true, не високосный - false). Каждый 4-й год является високосным, кроме каждого 100-го, при этом каждый 400-й – високосный.

6. Задать целочисленный массив, состоящий из элементов 0 и 1. Например: [1, 1, 0, 0, 1, 0, 1, 1, 0, 0]. С помощью цикла и условия заменить 0 на 1, 1 на 0;

7. Задать пустой целочисленный массив длиной 100. С помощью цикла заполнить его значениями 1 2 3 4 5 6 7 8 ... 100;

8. Задать массив [1, 5, 3, 2, 11, 4, 5, 2, 4, 8, 9, 1] пройти по нему циклом, и числа меньшие 6 умножить на 2;

Практическое задание



9. Создать квадратный двумерный целочисленный массив (количество строк и столбцов одинаковое), и с помощью цикла(-ов) заполнить его диагональные элементы единицами (можно только одну из диагоналей, если обе сложно). Определить элементы одной из диагоналей можно по следующему принципу: индексы таких элементов равны, то есть $[0][0]$, $[1][1]$, $[2][2]$, ..., $[n][n]$;

10. Написать метод, принимающий на вход два аргумента: `len` и `initialValue`, и возвращающий одномерный массив типа `int` длиной `len`, каждая ячейка которого равна `initialValue`.